

Task 8 — IDS Evasion (TryHackMe)

[TryHackMe](#) | [vinyxzeditz](#)

Final Report — Windows Host Variant (No screenshot placeholders)

Author: Manu raj (report compiled). Generated: 2025-10-18 17:59:11 (local).

Environment note: This exercise was completed from a Windows 10/11 laptop hosting VirtualBox. A Kali Linux attacker VM was used for offensive tooling; the target machine was imported from the provided OVA. All lab traffic was kept on a host-only VirtualBox network.

Executive Summary

This report documents the completion of the 'IDS Evasion' room on TryHackMe. The aim was to explore Network-based Intrusion Detection Systems (NIDS) behavior, test standard reconnaissance tools, and experiment with evasion techniques. The lab was run from a Windows laptop using VirtualBox (OVA import for target) and a Kali attacker VM. The experiments demonstrated that signature-based detection is limited against fragmentation, decoys, and idle scans, and highlighted the importance of IDS tuning and supplemental endpoint monitoring. The report contains step-by-step actions, observed results, and answers to the room questions. **Introduction**

Intrusion Detection Systems (IDS) monitor network traffic to detect suspicious activity. This room focused on NIDS behavior (Suricata in the lab), evasion techniques such as packet fragmentation, decoy and idle scans, timing adjustments and the interplay between NIDS and HIDS tools like Wazuh. The practical work involved running scans from Kali, observing alerts via the target's alerts web UI, and verifying HIDS alerts on the target where possible.

Setup and Tools

- Host OS: Windows 10/11 (desktop environment).
- Hypervisor: Oracle VirtualBox (imported target OVA).
- Attacker: Kali Linux VM (VirtualBox) with standard pentesting tools: nmap, gobuster, nikto, sqlmap, hydra.
- Target: Provided OVA running Suricata (IDS) and Wazuh (HIDS) with a web alerts interface at `http://MACHINE_IP:8000/alerts`.
- Network mode: VirtualBox Host-only adapter (vboxnet0) to keep lab traffic isolated from the real network.
- Optional host tooling: Windows built-in Snipping Tool / Snip & Sketch for screenshots, PowerShell for file transfers, and WSL2 if additional Linux tooling on the host was required.

Steps Performed (concise)

VirtualBox: Imported the provided OVA and configured a Host-only Adapter (vboxnet0). Took an snapshot named 'pre-attacks'.

Target boot: Started the OVA and verified services (SSH, HTTP, Grafana) using a baseline nmap from the Kali VM.

Baseline scanning: Performed a full SYN scan to establish which scans triggered Suricata alerts and to capture the baseline behavior.

Evasion testing: Performed fragmented scans (-f), decoy scans (--decoy), idle scans (-sl), and combined timing/fragmentation (-T2 -f). Monitored the alerts web UI after each attempt. 5) Web enumeration: Used gobuster and nikto to find interesting paths and content (e.g., /login, /admin).

HIDS checks: Observed Wazuh for host-level alerts after running post-exploitation and enumeration tools on the target where applicable.

Privilege escalation: Followed the hint path exposed by linPEAS and obtained the root flag in /root/ as the room requires (recorded flag: {SNEAK_ATTACK_CRITICAL}).

Observations and Results

Baseline SYN scans triggered Suricata alerts as expected; signature-based rules matched patterns consistent with Nmap scans.

Fragmented scans (-f) were less likely to trigger immediate signature alerts; fragmentation delayed or prevented matching by some rulesets.

Decoy scans showed multiple source IPs in the logs, diluting attribution and in some cases delaying analyst triage.

Idle scans using an appropriate zombie host provided the cleanest evasion—Suricata logs pointed to the zombie host rather than the attacker VM in those cases.

Timing adjustments (e.g., -T2) combined with fragmentation reduced rate-based alerts and anomaly flags.

Wazuh (HIDS) detected some post-exploitation actions and file additions (e.g., when linPEAS was added), producing higher-severity host alerts compared to network-only visibility. This illustrates the value of endpoint monitoring.

Answers to Room Questions (as submitted)

2 - Intrusion Detection Basics: What IDS detection methodology relies on rule sets?

Answer: Signature-based detection

3 - Network-based IDS (NIDS): What widely implemented protocol has an adverse effect on the reliability of NIDS?

Answer: TLS

Experiment by running tools against the target and viewing the resultant alerts. Is there any unexpected activity?

Answer: No

4 - Reconnaissance and Evasion Basics: What scale is used to measure alert severity in Suricata? (*-*)

Answer: 1-3

How many services is nmap able to fully recognise when the service scan (-sV) is performed?

Answer: 3

5 - Further Reconnaissance Evasion: Nikto, should find an interesting path when the first scan is performed, what is it called?

Answer: /login

What value is used to toggle denial of service vectors when using scan tuning (-T) in nikto?

Answer: 6

Which flags are used to modify the request spacing in nikto? Use commas to separate the flags in your answer.

Answer: 6,a,b

6 - Open-source Intelligence: What version of Grafana is the server running?

Answer: 8.2.5

What is the ID of the severe CVE that affects this version of Grafana?

Answer: CVE-2021-43798

If this server was publicly available, What site might have information on its services already?

Answer: Shodan

How would we search the site 'example.com' for pdf files, using advanced Google search tags?

Answer: site:example.com filetype:pdf

7 - Rulesets: What is the password of the grafana-admin account?

Answer: GraphingTheWorld32

Is it possible to gain direct access to the server now that the grafana-admin password is known? (yay/nay)

Answer: yay

Are any of the attached IDS able to detect the attack if the file /etc/shadow is requested via the exploit, if so what IDS detected it?

Answer: suricata

8 - Host Based IDS (HIDS): What category does Wazuh place HTTP 400 error codes in?

Answer: web

Play around with some post-exploitation tools and commands and make note of what activity is detected by Wazuh; compare it to the activity that's detected by Suricata.

Answer: No answer provided

9 - Privilege Escalation Recon: What tool does linPEAS detect as having a potential escalation vector?

Answer: docker

Is an alert triggered by Wazuh when linPEAS is added to the system, if so what its severity?

Answer: 5

10 - Performing Privilege Escalation: Perform the privilege escalation and grab the flag in /root/

Answer: {SNEAK_ATTACK_CRITICAL}
rded(root_flag.txt).

Conclusions & Recommendations

- Signature-based NIDS (like Suricata with default rules) reliably detect noisy scans but can be evaded by fragmentation, decoy/idle scans, and careful timing.
- Use a layered approach: endpoint agents (HIDS/EDR), flow analysis, and threat intelligence to detect tooling that evades network signatures.
- Maintain frequent rule-tuning and baseline network behavior for each environment. Implement TLS inspection judiciously where legal/appropriate.

Appendix: Commands used (I used often to learn and discover till this - commands copy/paste)

```
nmap -sV -p- -A TARGET_IP -oN scan_full.txt nmap -f -sS TARGET_IP -oN  
nmap_fragmented.txt nmap -sS -p 80 --decoy "ME,1.2.3.4,5.6.7.8"  
TARGET_IP nmap -sl ZOMBIE_IP TARGET_IP nmap -f -T2 -p 80  
TARGET_IP gobuster dir -u http://TARGET_IP -w  
/usr/share/wordlists/dirbuster/directorylist-2.3-medium.txt -o  
gobuster_admin.txt nikto -h http://TARGET_IP -output nikto_out.txt sudo tail  
-n 200 /var/log/suricata/eve.json > eve.
```



